

AI Operations (AIOps) — Module 3 Assignment

Infrastructure & Containerization — Written Write-up

Student: DA24B037 — Application: Spam-Detection API (TF-IDF + Naive Bayes)

Q1 Single-Stage vs. Multi-Stage Docker

1 & 2. Image sizes

The naive single-stage build (`Dockerfile.naive`), based on the full `python:3.11` image, produced an image of **2.93 GB**. The multi-stage build (`Dockerfile.multistage`) produced an image of **782 MB** — a reduction of approximately **73.3%**.

3. Why the multi-stage image is smaller (specific to this image)

The builder stage uses the full `python:3.11` image because compiling scikit-learn's and numpy's C extensions during `pip install` requires `gcc` and system build headers present in the full Debian-based image but absent from the slim variant. This stage also holds `spam_dataset.csv` and `train.py`, used only to produce `model.joblib`. The final stage starts fresh from `python:3.11-slim` and copies over only three artifacts via `COPY --from=builder`: the installed site-packages (`/root/.local`), `main.py`, and `model.joblib`. Left behind in the builder stage — and therefore absent from the final image — are the `gcc/build-essential` toolchain, `apt` package lists and caches from the full base image, `pip`'s download cache, the raw training dataset, and the training script itself, none of which are needed at runtime.

Q2 Multi-Container Orchestration with Docker Compose

1. Caching behaviour

The `/predict` endpoint checks Redis for the key `predict:<exact input text>` before running inference. On a cache miss the pipeline computes the label, stores it in Redis via `SETEX` with a 300-second TTL, and returns it. On a cache hit the cached label is returned directly, skipping TF-IDF + Naive Bayes inference entirely.

2. docker-compose.yml

Two services are defined: `api` (built from `Dockerfile.multistage`) and `cache` (`redis:7-alpine`). The API reaches Redis via the hostname `cache`, resolved automatically by Compose's internal DNS on the default project network — no manual linking or IP configuration required.

3. Evidence of cache-hit speedup

Two identical POST requests were timed with `curl -w "%{time-total}":`

```
First call (cache miss, computes prediction): 0.022910 s
Second call (cache hit, returns cached value): 0.001942 s
Speedup: approximately 11.8x faster on the cache hit.
```

This was confirmed directly against Redis: `redis-cli KEYS "predict:*"` listed the cached key, and `redis-cli GET` on it returned the correct cached label ("spam"), verifying the value was stored and retrieved correctly.

4. Compose vs. Kubernetes

Docker Compose coordinates multiple interdependent containers (the API and Redis) as a single declarative unit on **one host** — defining build steps, environment variables, inter-container networking, and startup order without manually issuing several `docker run` commands. Its scope is single-host, multi-container orchestration for local development and simple deployments. Kubernetes, in contrast, orchestrates containers across a **cluster of multiple machines**, handling scheduling across nodes, horizontal scaling, self-healing, and rolling updates — an operational scale Compose does not attempt to address.

Q3 Kubernetes Indexed Job: Parallel Data Validation

1. Parallelism choice and justification

Given the assignment's stated constraint of 2 nodes with 2 allocatable CPUs each (4 total), each validation pod was given a CPU request/limit of 500m (0.5 CPU) — conservative for lightweight CSV parsing and regex validation. With `parallelism: 4`, four concurrent pods consume 2 of the 4 total CPUs, leaving headroom for per-node system daemons and avoiding pods stalling in Pending. `completions: 8` matches the 8 shards, so the batch completes in two waves of 4.

2. Evidence parallelism was reached

After applying the Job, `kubectl get pods -o wide` showed pods 0–3 with matching ages (started together as the first wave) and pods 4–7 starting a few seconds later once the first four completed — confirming 4 pods genuinely ran concurrently. Pods were scheduled across both cluster nodes (`minikube` and `minikube-m02`).

3. Result collection and shared-volume justification

Results were collected via `kubectl logs` against each pod (the Kubernetes API), not a shared volume. `minikube`'s default storage provisioner backs PersistentVolumes with local host-path storage on a single node, which does not support safe concurrent multi-pod read/write access — risking race conditions across 8 independent shard-validation tasks. Reading results through pod logs avoids this entirely. `JOB_COMPLETION_INDEX` (set automatically for Indexed Jobs) told each pod which shard to load; the Downward API (`spec.nodeName`, `metadata.name`) exposed node and pod name in each log line.

```
Shard 0: node=minikube-m02, rows=50, invalid=8   Shard 1: node=minikube, rows=50, invalid=8
Shard 2: node=minikube-m02, rows=50, invalid=12  Shard 3: node=minikube-m02, rows=50, invalid=9
Shard 4: node=minikube-m02, rows=50, invalid=14  Shard 5: node=minikube, rows=50, invalid=7
Shard 6: node=minikube-m02, rows=50, invalid=7   Shard 7: node=minikube-m02, rows=50, invalid=14
Total: 400 rows validated, 79 invalid rows found.
```

4. Re-justification for a 3-node / 6-CPU cluster

With 6 total CPUs instead of 4, parallelism would be increased from 4 to 6. At 500m CPU per pod, 6 concurrent pods consume exactly 6 CPUs — fully utilizing the larger cluster without over-committing, while preserving the same safety-margin ratio for system daemons. Since there are only 8 completions total, running 6 in parallel finishes the batch in two waves (6, then 2) instead of two even waves of 4, reducing total wall-clock time.

Q4 Kubernetes Deployments: Self-Healing & Rolling Updates

1. Manifests

A Deployment with `replicas: 2`, CPU/memory requests and limits, and a `readinessProbe` against `/healthz` was applied alongside a matching NodePort Service exposing port 8000. Both applied successfully and both pods reached `1/1 Running`, scheduled across the two cluster nodes.

2. Self-healing

One running pod was deleted manually with `kubectl delete pod`. `kubectl get pods` immediately after showed a brand-new pod (different random suffix) appear within seconds, progressing from `ContainerCreating` to `1/1 Running`, while the untouched original pod remained unaffected — the replica count stayed at exactly 2 throughout. The **ReplicaSet controller** managed by the Deployment is responsible: it runs a continuous reconciliation loop comparing the actual number of running pods matching its label selector against the desired replica count. When a pod disappears, the observed count falls below the desired count, and the controller immediately creates a replacement — it does not wait for any external signal, only its own periodic reconciliation.

3. Rolling update

A `version` field was added to `/healthz`, the image rebuilt and tagged `spam-api:v2`, loaded into the cluster with `minikube image load`, and applied with `kubectl set image`. `kubectl rollout status` reported the update proceeding one pod at a time ("1 out of 2 new replicas have been updated" → "1 old replicas are pending termination") before finishing with "successfully rolled out", and `kubectl rollout history` showed two revisions. Throughout, at least one pod (old or new) was always `Running` and ready, and the live endpoint was verified afterward to return `{"status": "ok", "version": "v2"}`, confirming the new code was genuinely serving traffic. Had this application been deployed as a Job instead, there would be no update mechanism at all: a Job runs pods to completion once and has no continuously-reconciled desired state or live label-selected service. Deploying a new version would require deleting and recreating the Job entirely, causing a full outage rather than the incremental, zero-downtime replacement a Deployment performs automatically.

4. Deployment vs. Job — fit for workload shape

A Deployment suits long-running, always-available services like this API: it treats a fixed number of replicas as a steady state to continuously maintain, self-healing failures and rolling out changes without downtime, since the workload is expected to keep serving requests indefinitely. A Job suits batch, run-to-completion work like the shard validation in Q3: each pod is expected to finish and exit successfully exactly once, there is no ongoing state to reconcile, and the controller simply tracks whether the required number of completions has been reached.